Artificial Intelligence Software, Lab 1

Intro to ROS 2:
Concepts, Architecture, Tools

Center for
Cognitive
Modeling

What is ROS?

More than just software

Definition: ROS (Robot Operating System) is a set of software libraries and tools that help you build robot applications.

- **Plumbing (Middleware):** Message passing, hardware abstraction (HAL).
- **Tools:** GUI (RViz, RQT), CLI, Build systems (Colcon).
- **Capabilities:** Navigation (Nav2), Manipulation (MoveIt), Perception.
- **Ecosystem:** Thousands of packages, huge community.

"Don't reinvent the wheel. Focus on your specific algorithm."

History: ROS 1 vs ROS 2

Why the change was necessary

ROS 1 (2007)

- **Master Node:** Centralized (Single Point of Failure).
- **TCP/UDP:** Custom protocol (TCPROS).
- **Research only:** Not real-time safe.
- **Linux only:** Hard to port.
- **Spaghetti codebase:** Hard to maintain (monolithic).

ROS 2 (2017)

- **DDS:** Decentralized Data Distribution Service (Industry standard).
- **Real-time:** Support for embedded hard RT.
- **Security:** SROS2 (Encryption).
- **Cross-platform:** Linux, Windows, macOS, RTOS.

Pros & Cons in Production

Why it is the standard

Why industry uses it (Pros):

- **Modularity:** Swap a camera driver, and the rest of the code works.
- **Tools:** RViz and RQT save 1000s of developer hours.
- **Standardization:** Integrating 3rd party robots is easier.

Where it hurts (Cons):

- **Complexity:** Steep learning curve (DDS configuration, QoS).

Real World Examples

Where ROS 2 is inside

Autonomous Driving (Autoware):
Self-driving cars stack.
Uses: LiDAR, Mapping, Planning.

Logistics (Nav2):
Warehouse robots.
Uses: Path planning, obstacle avoidance.

Manipulation (MoveIt 2):
Robot arms.
Uses: Inverse Kinematics, Collision Checking.

References: Introduction

EN Why ROS 2? (Design Docs)

EN Autoware (Autonomous Driving)

EN Nav2 (Logistics)

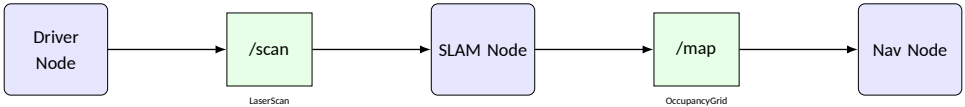
EN MoveIt 2 (Manipulation)

The Computation Graph

Overview

ROS acts as a peer-to-peer network (Graph).

- **Node:** A participant in the graph.
- **Edge:** The communication channel (Topic, Service, Action).



Parameters

Node Configuration

Definition: Parameters are configuration values (int, float, bool, string) stored inside a node.

CLI Interactivity:

- ros2 param list: View all.
- ros2 param get <node> <key>
- ros2 param set <node> <key> <val>
- ros2 param dump <node>

YAML Configuration:

```
/turtlesim:
  ros__parameters:
    background_r: 0
    background_g: 255
```

Load: ... --ros-args --params-file config.yaml

Topics vs Services vs Actions in ROS

Topics (Publish/Subscribe)

Communication type: Asynchronous, many-to-many
Use case: Continuous data streams
Blocking: Non-blocking
Example: Lidar publishes /scan, controller subscribes
Typical data: Sensor data, velocity commands (/cmd_vel)

Services (Request/Response)

Communication type: Synchronous, one-to-one
Use case: Quick, finite operations
Blocking: Client waits for response
Example: Call /reset_odom to reset odometry
Typical data: Configuration, state queries

Actions (Goal/Feedback/Result)

Communication type: Asynchronous, long-running tasks
Use case: Tasks with progress and cancellation
Blocking: Non-blocking with feedback
Example: Send navigation goal to /navigate_to_pose, receive feedback on distance remaining
Typical data: Navigation, manipulation tasks

References: Core Concepts

EN Understanding Nodes (Docs)

EN Topics (Docs)

EN Services (Docs)

EN Actions (Docs)

Workspace Structure

Where code lives

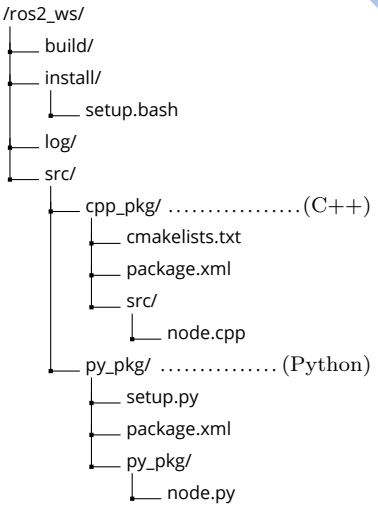
The workspace is just a folder with a specific structure.

- src/: Source code (packages). **Only edit this!**
- build/: Intermediate build files (CMake cache).
- install/: Final artifacts (executables, setup files).
- log/: Logs from compilation and running.

Dependency Management:

rosdep install --from-paths src -y --ignore-src

Installs system libraries (e.g. OpenCV, Boost) required by your packages.



Colcon Build System

The Universal Build Tool

colcon iterates over packages in topological order.

Common command:

```
colcon build --symlink-install
```

- **–symlink-install:** Critical for Python dev. It symlinks python files from src to install. You can change code and run without rebuilding!
- Without this, you must run colcon build after every single change.

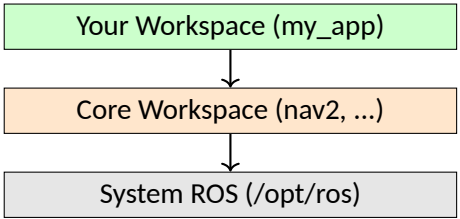
Sourcing: Underlay vs Overlay

Underlay: source /opt/ros/humble/setup.bash

Sets basic environment variables (PATH, PYTHONPATH).

Overlay: source ~/ros2_ws/install/setup.bash

Prepends your workspace paths to the system paths.



Chaining Workspaces

References: Environment

EN Creating a Workspace

EN Colcon Documentation

EN Managing Dependencies with rosdep

Package Anatomy

The Atomic Unit of ROS

A package is a container for your ROS code. It must contain package.xml.

Python Package (ament_python):

- package.xml: Defines dependencies (‘rclpy‘, ‘std_msgs‘).
- setup.py: Entry points (link executable name → python function).
- resource/: Empty file for index.

Creation Command:

```
ros2 pkg create --build-type ament_python my_pkg
```

setup.py: The Entry Point

How to tell ROS ”run this python function”?

```
entry_points={
    'console_scripts': [
        # executable_name = package.module:function
        'talker = my_pkg.publisher_member_function:main',
        'listener = my_pkg.subscriber_member_function:main',
    ],
},
```

After building, you can run: ros2 run my_pkg talker

Making Files Visible: data_files

Why can't I see my launch/yaml files?

Problem: By default, setup.py only installs Python files. Other files (launch, config, maps) are ignored and won't be found by ros2 run/launch.

Solution: You must explicitly install them into the share directory using data_files in setup.py.

```
from glob import glob
import os
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),

        (os.path.join('share', package_name, 'launch'),
         glob(os.path.join('launch', '*launch.*'))),
    ],
# ...
```

Dependencies

package.xml

Important for release and rosdep.

```
<package format="3">
  <name>my_pkg</name>
  <description>Examples</description>
  <maintainer email="me@email.com">Me</maintainer>
  <license>Apache-2.0</license>
  <depend>rclpy</depend>
  <depend>std_msgs</depend>
  <test_depend>ament_copyright</test_depend>
  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

Resolving Dependencies

The rosdep Magic

How to install system binaries (apt/pip) defined in package.xml?

1. **Define:** Add <depend>box2d</depend> in package.xml.
2. **Install:** Run rosdep in the workspace root.

```
rosdep install --from-paths src -y --ignore-src
```

- **–from-paths src**: Look inside src/ folder for package.xml files.
- **-y**: ”Yes” to all confirmations.
- **–ignore-src**: Don’t try to install a package if it’s already in src/ (source code takes precedence).

Standard Workflow

Step-by-step routine for running your node.

1. **Go to Root:** Always run build from workspace root.
   cd ~/ros2_ws
2. **Build:** Compile packages if changed.
   colcon build --symlink-install
3. **Source:** Load the overlay environment.
   source install/setup.bash
4. **Run:** Execute package node.
   ros2 run my_pkg my_node

Note: If you use –symlink-install, step 2 is only needed when adding new files or changing dependencies/setup.py. Pure python logic changes often work without rebuild.

References: Packages

EN Creating Your First Package

CLI Tools

First line of defense

Before opening GUI, check the terminal.

- ros2 topic list: What data is available?
- ros2 topic hz /scan: Is the sensor published? at what rate?
- ros2 topic echo /scan: Show raw data values.
- ros2 run <pkg><exec>: Run a node.
- ros2 param list: See configuration options.

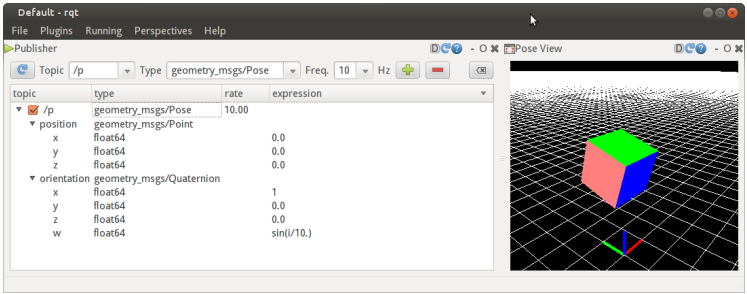
RQT: Graph & Console

Inspector of the Graph

When to use: Debugging connections, message flow, logs.

Plugins:

- **Graph:** Who is talking to whom?
- **Plot:** Visualize scalar data (speeds, battery) over time.
- **Service Caller:** Manually trigger services.

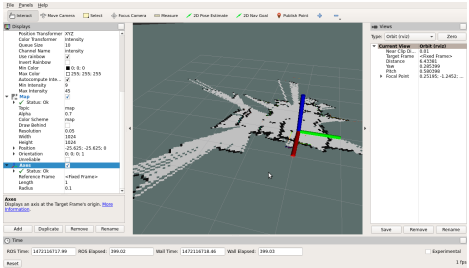


RViz 2: 3D Visualization

The Robot's Eye

When to use: Viewing spatial data. ”What does the robot see?”

- **TF (Transforms):** Coordinate frames relationship (Base → Camera).
- **Displays:** LaserScan, PointCloud2, RobotModel (URDF), Map, Odometry.



References: Tools

EN Turtlesim & RQT

EN RViz (ROS1 Wiki but relevant concepts)

What is a Rosbag?

The Black Box

Concept: A rosbag records the *exact* stream of messages passing through topics.

- **Serialization:** Messages are stored in binary format (CDR).
- **Storage:**
  - **SQLite3:** Default. Single file (or split). Good for compatibility.
  - **MCAP:** New standard. Optimized for robotics (indexing, attachments).

Bags vs Datasets

Raw vs Cooked

Rosbag (Raw Log)

- Contains EVERYTHING (timing, latency).
- Unstructured (just topics).
- Used for: *Simulated Replay*, Debugging ”why did it crash?”.

Dataset (Curated)

- Extracted from bags (Images, CSV).
- Annotated (Labels added).
- Used for: *Machine Learning Training*.

Workflow & Tools

1. **Record:** ros2 bag record -o output_name /image /scan
2. **Info:** ros2 bag info output_name (Duration, Size, Message count).
3. **Visualization:**
   - **Foxglove Studio:** Modern web-based visualizer for MCAP/Bags.
   - **PlotJuggler:** Great for analyzing time-series data.

References: Data

EN Recording and Playback (Official)

EN MCAP File Format

EN Foxglove Studio

Launch System

Writing scripts to run scripts

Imagine you have 10 nodes. You don’t want 10 terminals.

ROS 2 Launch (Python):

- **Introspection:** It’s python code, so you can use logic (if/else).
- **Events:** Restart node if it crashes.
- **Composition:** Include other launch files (‘include_launch_description‘).

Example Launch File

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='my_pkg',
            executable='talker',
            name='my_talker',
            parameters=[{'speed': 5.0}],
            remappings=[('/topic', '/
                chatter')]
        ),
        Node(
            package='my_pkg',
            executable='listener',
            name='my_listener'
        )
    ])
```

References: Launch

EN Launch System (Official)

EN Python vs XML vs YAML Launch

General References

- Autoware class about ROS2
- Intro to ROS2 course
- ROS2 курс на степике (русский)

Deserve “A” grade!

– Oleg Bulichev

bulichev.ov@mipt.ru

@Lupasic

Цифра, 521